Spring Boot – SOLID Principles

Gregory Galli
Teacher at Université Côte d'Azur (UniCA)

UNIVERSITÉ CÔTE D'AZUR

MIAGE RÉSEAU DES MIAGE DE FRANCE Nice

UNIVERSITÉ
CÔTE D'AZUR

Introduction

What is Spring Framework ?

UNIVERSITÉ CÔTE D'AZUR

- Open source
  - Sources available
    https://github.com/spring-projects/spring-framework
  - License : Apache 2
    - Modify
    - Distribute
    - Sublicense
    - Commercial Use
    - Summary : You can do what you like with the software, as long as you include the required notices
  - Documentation
    https://docs.spring.io/spring-framework/docs/current/spring-framework-reference/

What is Spring Framework ?

UNIVERSITÉ CÔTE D'AZUR

- Lightweight
  - Code is quite simple
    - Not need to extend classes or implement interfaces in most cases
    - Quite like POJO
  - Light overhead
    - Jar size qui reasonable

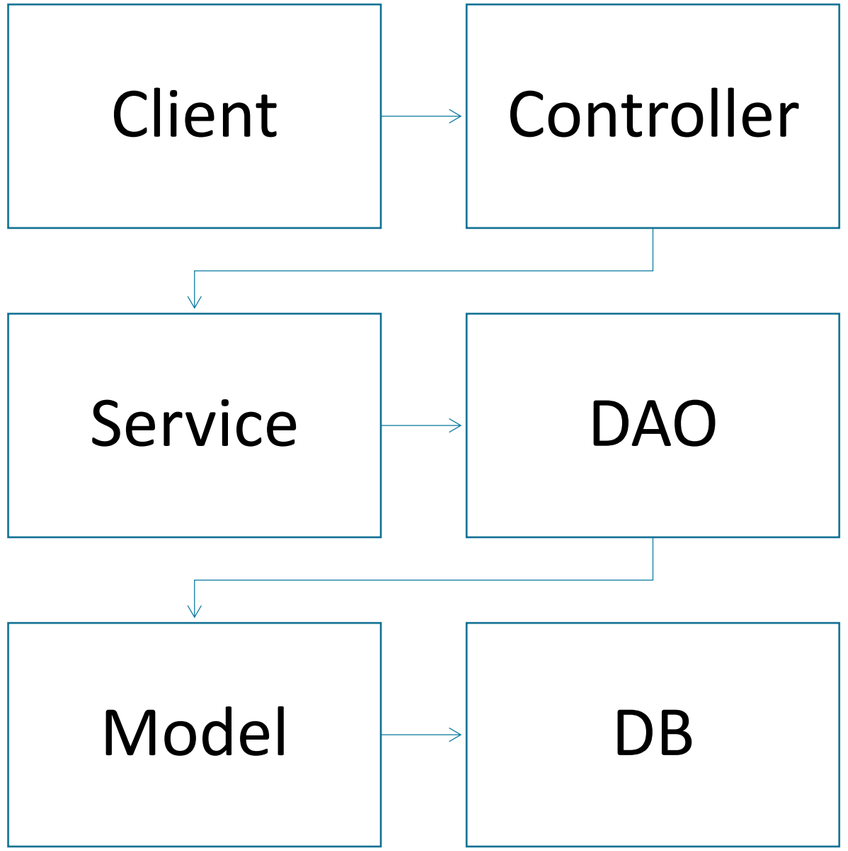
What is Spring Framework ?

UNIVERSITÉ CÔTE D'AZUR

- Easy access to a large variety of technologies
  - Database : JDBC, ORM / JPA, NoSQL
  - Security
  - Serialization : XML / JSON Marshalling
  - Accessibility : REST and SOAP Services
  - And many more
- Access via framework class instead
  - higher level access

What is Spring Framework ?

- Layered architecture
  - Parts do not have to worry about each others
  - Any part can be switched easily
- Model Layer : Entities
- DAO Layer : Repositories
- Service Layer : Business logic
- Controller Layer
  - Views
  - APIs



Why Spring Framework ?

UNIVERSITÉ CÔTE D'AZUR

- 1995 – Java
- 1999 – J2EE
  - JSP : dynamic web pages
  - EJB : persistence, transactions, security
  - Code was a pain
  - Testing was a pain
  - Heavy resources needed to run
  - Expensive from end to end

Why Spring Framework ?

UNIVERSITÉ CÔTE D'AZUR

- 2004 – Spring Framework 1.0
  - Dependency injection
  - Aims to keep the POJO side
  - Easy configuration
  - Quickly became a strong alternative to J2EE
- […]
- 2025 – Spring 6.2.11
  - Java 17 +, Jakarta EE9 +

Spring Framework

UNIVERSITÉ CÔTE D'AZUR

- Good bet for Enterprise grade applications
- No more just a J2EE alternative
- Huge community
  - Support
  - Recruiting
- Keep evolving and innovating

What is Spring Boot ?

UNIVERSITÉ CÔTE D'AZUR

- Framework to help create production-grade Spring-based applications
- Opinionated view of Spring & third-party libraries
- Embedded application server
- Aims for a quick start
- Very few configuration
- Automatic configuration (sometimes)

Why Spring Boot ?

UNIVERSITÉ CÔTE D'AZUR

- Provide a radically faster and widely accessible experience for all Spring dev
- Be opinionated by default but let the requirements take over quickly
- Embed wide range of non-functional features
  - Embedded servers
  - Security
  - Metrics
  - Externalized configuration
- No code generation needed
- No XML configuration

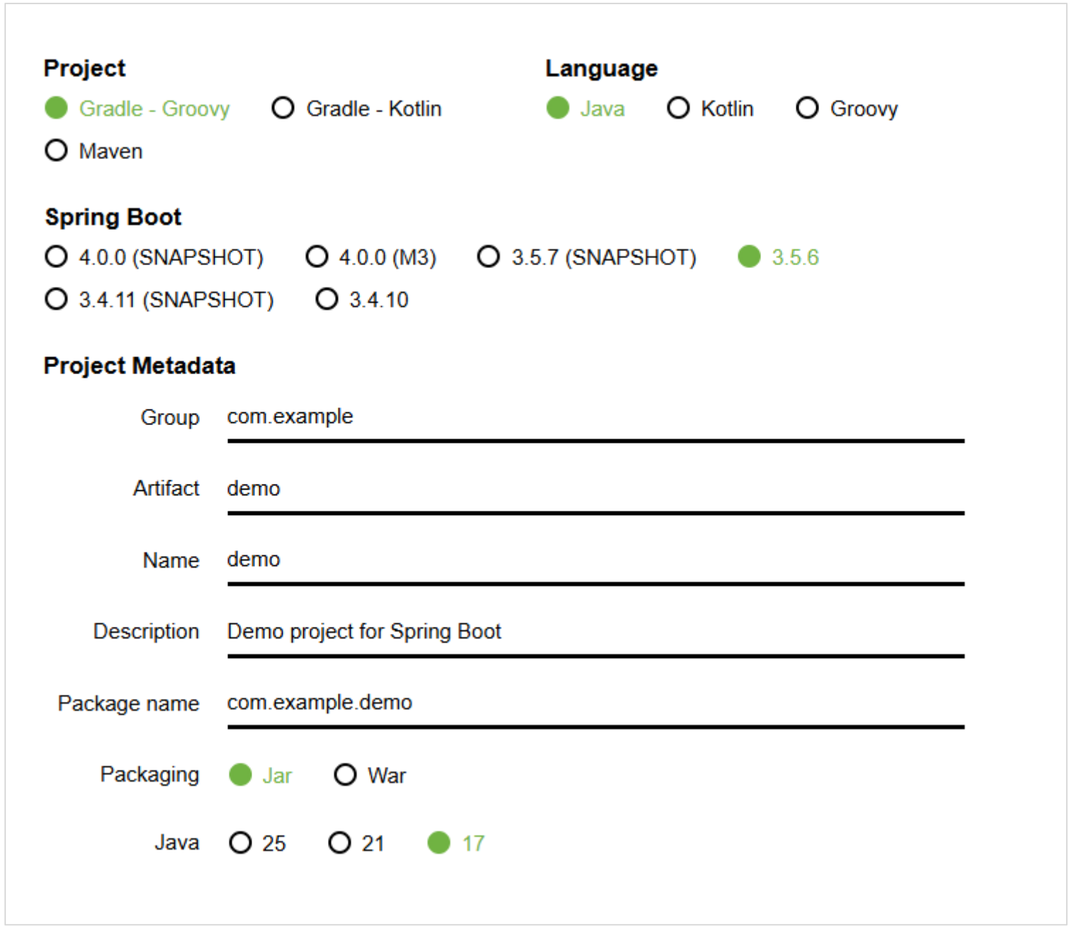
Spring Boot

- Dependencies
  - Maven 3.6.3 +
  - Gradle 7.6.4 +
- Application servers
  - Tomcat
  - Jetty
  - Undertow
- Compatible with Java version 17 to 25

Spring Initializr

- Used to generate Spring Boot projects : https://start.spring.io/
  - Versioning type
    - Maven
    - Gradle
  - Language
    - Java
    - Kotlin
    - Groovy
  - Framework version
  - Dependencies

Project configuration



UNIVERSITÉ
CÔTE D'AZUR

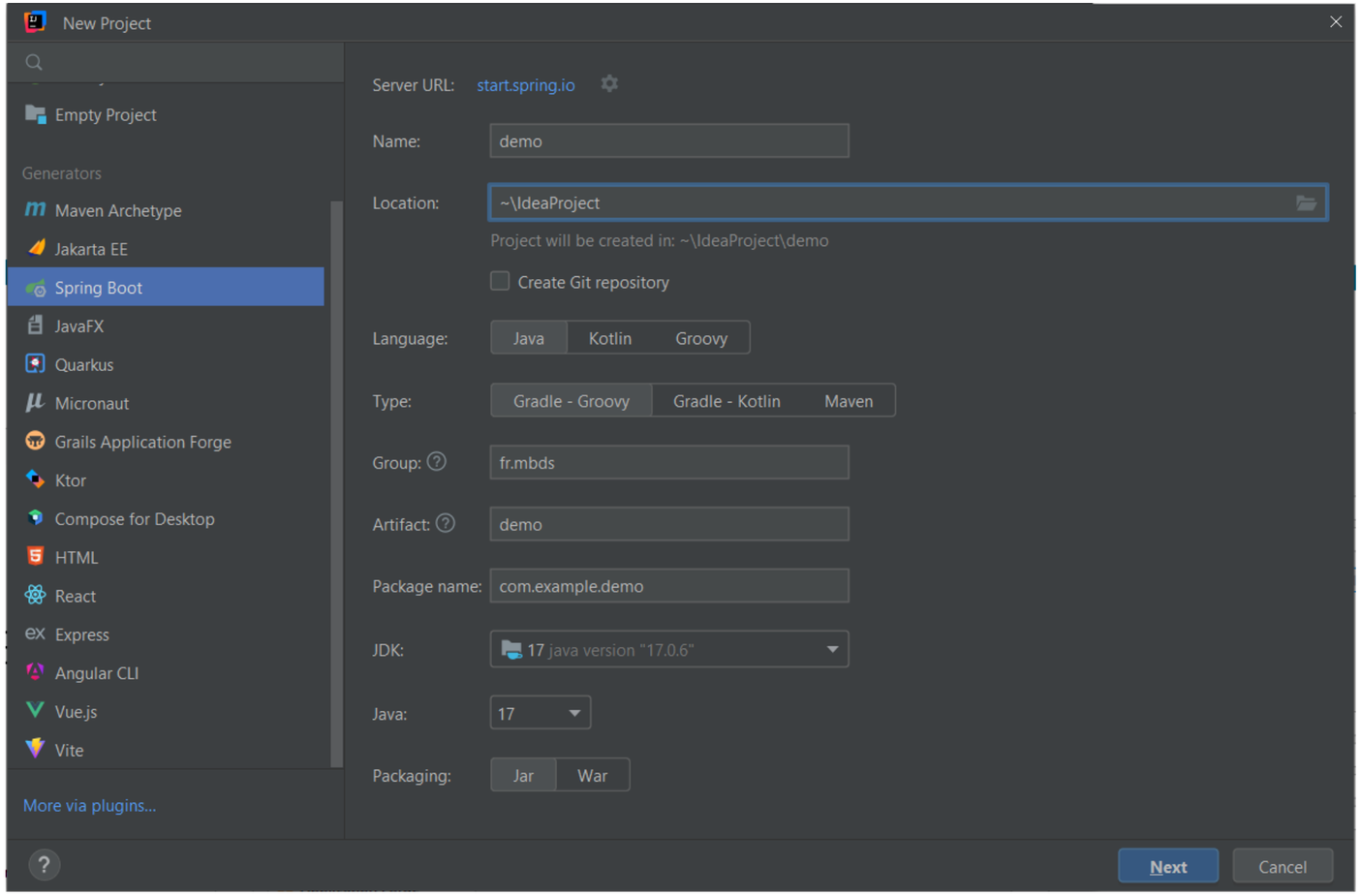
Project setup

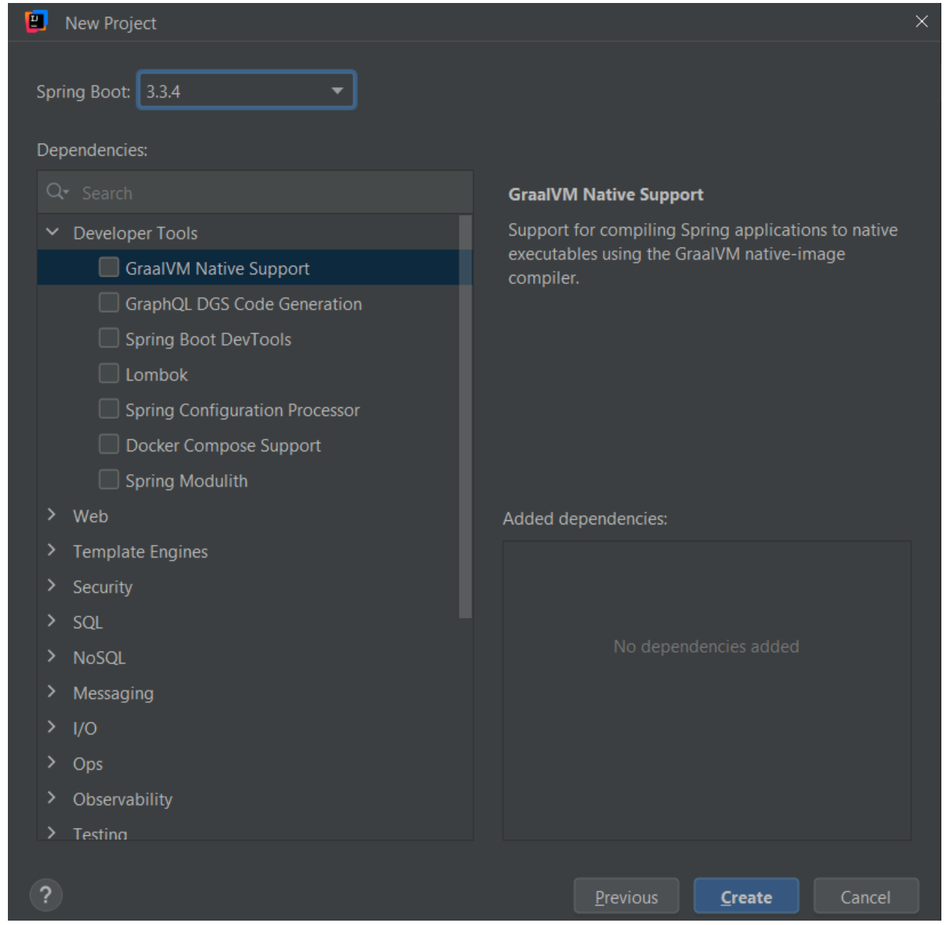
Spring Boot project setup

Setup

- IntelliJ IDEA Ultimate
  https://www.jetbrains.com/fr-fr/idea/download/
- Academic license
  https://www.jetbrains.com/fr-fr/community/education/#students
- Install and start IntelliJ

Setup


New Project
Empty Project
Generators
Maven Archetype
Jakarta EE
Spring Boot
JavaFX
Quarkus
Micronaut
Grails Application Forge
Ktor
Compose for Desktop
HTML
React
Express
Angular CLI
Vue.js
Vite
More via plugins...
Server URL: start.spring.io
Name: demo
Location: ~\IdeaProject
Project will be created in: ~\IdeaProject\demo
Create Git repository
Language: Java Kotlin Groovy
Type: Gradle - Groovy Gradle - Kotlin Maven
Group: fr.mbds
Artifact: demo
Package name: com.example.demo
JDK: 17 java version "17.0.6"
Java: 17
Packaging: Jar War
Next
Cancel


New Project
Spring Boot: 3.3.4
Dependencies:
Search
Developer Tools
GraalVM Native Support
GraphQL DGS Code Generation
Spring Boot DevTools
Lombok
Spring Configuration Processor
Docker Compose Support
Spring Modulith
Web
Template Engines
Security
SQL
NoSQL
Messaging
I/O
Ops
Observability
Testing
GraalVM Native Support
Support for compiling Spring applications to native executables using the GraalVM native-image compiler.
Added dependencies:
No dependencies added
Previous
Create
Cancel

Setup

- Dependencies will be downloaded
- Enable Auto-Import if asked
- Exceptions auto-configuration to firewall / virus protection software

Setup

```java
// SpringbootApplication.java
package com.course.springboot;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@SpringBootApplication
public class SpringbootApplication {

    @RequestMapping("/")
    String home() {
        return "Hello World !";
    }

    public static void main(String[] args) {
        SpringApplication.run(SpringbootApplication.class, args);
    }
}
```

Setup (Annotations)

- @SpringBootApplication encapsulate three other annotations
  - @Configuration
    - Allow to register extra beans in the context or import additional configuration classes
  - @ComponentScan
    - Enable @Component scan on the package
  - @EnableAutoConfiguration
    - Enable Spring Boot’s auto configuration mechanism
    - Will generate automatic configuration for the declared dependencies
    - Will attempt to guess and configure beans you are likely to need

Setup : application.properties

- Can specify a few things through properties files
  - src/main/ressources/application.properties
- Server port
- Application name

```
#src/main/resources/application.properties

server.port = 8091
spring.application.name = course
```

Setup : application.yml

- Can also use a YAML file
- Syntax is a lot more appropriate in some cases
  - If a lot of configuration for a specific package
  - If you need to define arrays of values

```
#application.yml

servers:
  - dev.example.com
  - local.example.com
  - test.example.com
```

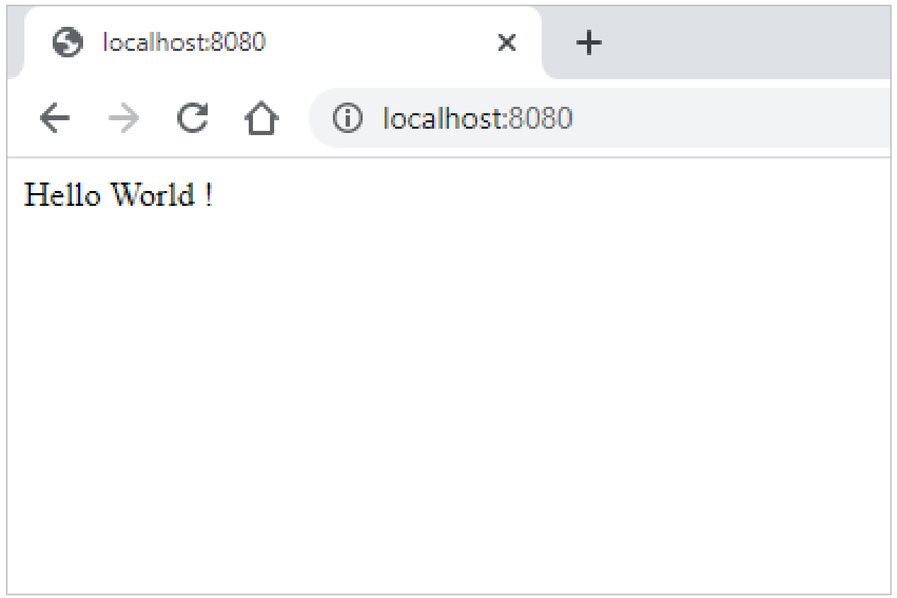
Setup

- Run project and open specified port on localhost
  - http://localhost:8080/ by default

```
2020-09-10 17:36:55.778  INFO 15752 --- [  restartedMain] o.s.b.w.embedded.tomcat.TomcatWebServer  : Tomcat initialized with port(s): 8080 (http)
2020-09-10 17:36:55.784  INFO 15752 --- [  restartedMain] o.apache.catalina.core.StandardService   : Starting service [Tomcat]
2020-09-10 17:36:55.784  INFO 15752 --- [  restartedMain] org.apache.catalina.core.StandardEngine  : Starting Servlet engine: [Apache Tomcat/9.0.37]
2020-09-10 17:36:55.832  INFO 15752 --- [  restartedMain] o.a.c.c.C.[Tomcat].[localhost].[/]       : Initializing Spring embedded WebApplicationContext
2020-09-10 17:36:55.832  INFO 15752 --- [  restartedMain] w.s.c.ServletWebServerApplicationContext : Root WebApplicationContext: initialization completed in 594 ms
2020-09-10 17:36:55.928  INFO 15752 --- [  restartedMain] o.s.s.concurrent.ThreadPoolTaskExecutor  : Initializing ExecutorService 'applicationTaskExecutor'
2020-09-10 17:36:56.022  INFO 15752 --- [  restartedMain] o.s.b.d.a.OptionalLiveReloadServer       : LiveReload server is running on port 35729
2020-09-10 17:36:56.046  INFO 15752 --- [  restartedMain] o.s.b.w.embedded.tomcat.TomcatWebServer  : Tomcat started on port(s): 8080 (http) with context path ''
2020-09-10 17:36:56.053  INFO 15752 --- [  restartedMain] c.c.springboot.SpringbootApplication     : Started SpringbootApplication in 1.081 seconds (JVM running for 1.63
2020-09-10 17:37:02.005  INFO 15752 --- [nio-8080-exec-1] o.a.c.c.C.[Tomcat].[localhost].[/]       : Initializing Spring DispatcherServlet 'dispatcherServlet'
2020-09-10 17:37:02.005  INFO 15752 --- [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet        : Initializing Servlet 'dispatcherServlet'
2020-09-10 17:37:02.009  INFO 15752 --- [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet        : Completed initialization in 4 ms
```

Setup

- Project is running !



- Have a look at build.gradle

Dependencies

UNIVERSITÉ CÔTE D'AZUR

- Declared into the build.gradle / pom.xml file
- Dependencies version is optional
- Spring will manage dependencies versions
- Can specify a specific version though
  - Can be useful if you discover some incompatibilities
- Learn more about gradle
  https://docs.spring.io/spring-boot/gradle-plugin/index.htm
- Learn more about maven
  https://docs.spring.io/spring-boot/maven-plugin/index.html

Starters

- Set of dependency descriptors
  - Contains packs of dependencies
- Easy to setup specific aspects
- Name start with “spring-boot-starter”
- Eg: « spring-boot-starter-web » contains multiple packages :
  - Web building
  - Rest
  - Spring MVC
  - Embedded Tomcat
  - …
- Full list at https://github.com/spring-projects/spring-boot/tree/main/starter

Configuration

- Done through Java-based configuration
- Can be defined with XML sources but not recommended
- `@Configuration` annotation for defining a class as a configuration class
- `@Import` annotation available to import additional configuration classes
- `@ComponentScan` will get all components as well as configuration classes

Auto configuration

- Will try to automatically configure the Spring Application
  - Based on the dependencies added
- Eg: Declaring a dependency towards a DBMS without specifying any database connection
  - Auto configuration will use an in-memory database by default
- Aims to be non invasive
  - Use default auto configuration when you start a project
  - Replace and override when you need specific configuration
- To check how the current configuration is resolved, launch project with debug mode

UNIVERSITÉ
CÔTE D'AZUR

Components & Dependency injection

Components, Autowire

What is a Component ?

- Most generic Spring annotation
- Any class with `@Component` annotation will be registered as a Spring Bean
- `@ComponentScan` will discover all Components in
  - Current package
  - Subpackages
- Make the class available for dependency injection

What is a Component ?

- Other specialized Component
  - `@Service`
  - `@Repository`
  - `@Controller`
  - …
- Component or specialized components annotations are called **Stereotypes**

Specialized Components

- `@Service`
  - Class will provide some services
  - Utility class : Business logic
- `@Repository`
  - Declare a class dealing with CRUD operations
  - Used for DAO implementation to handle database communication
- `@Controller`
  - Used with web applications / REST web services
  - Specify the class is a controller in charge of user request handling

@Autowired ?

UNIVERSITÉ CÔTE D'AZUR

- Annotation-driven Dependency Injection since Spring 2.5
- `@Autowired` is the main annotation of this feature
- Allows Spring to resolve and inject collaborating beans into our bean
- `@Autowired` can be used on
  - Fields
  - Setters
  - Constructors

Field based injection

UNIVERSITÉ CÔTE D'AZUR

- Eliminates the need for getters and setters

```
@Component
public class Runner implements CommandLineRunner {

    @Autowired
    private RandomNameGenerator randomNameGenerator;

    @Override
    public void run(String... args) throws Exception {
    }
}
```

Setter based injection

UNIVERSITÉ CÔTE D'AZUR

- Autowired Setter will be called after the « holders » is created

```
@Component
public class Runner implements CommandLineRunner {

    private RandomNameGenerator randomNameGenerator;

    @Autowired
    public void setRandomNameGenerator(RandomNameGenerator randomNameGenerator)
    {
        this.randomNameGenerator = randomNameGenerator;
    }

    @Override
    public void run(String... args) throws Exception {
    }
}
```

Constructor based injection

- Can declare the injected fields final (only solution for immutable/final)

```
@Component
public class Runner implements CommandLineRunner {

    private final RandomNameGenerator randomNameGenerator;

    @Autowired
    public Runner(RandomNameGenerator randomNameGenerator) {
        this.randomNameGenerator = randomNameGenerator;
    }

    @Override
    public void run(String... args) throws Exception {
    }
}
```

Dependency Injection

- `@Autowired` is the Spring framework equivalent of `@Inject` (JSR-330)
- Execution path for setter and field injection is as follow
  - Match by Type
  - Match by Qualifier
  - Match by Name
- Let’s see a few examples about this

Dependency Injection – By Type

```java
public class Runner implements CommandLineRunner {

    private RandomNameGenerator randomNameGenerator;

    @Autowired
    private RandomNumberGenerator randomNumberGenerator;

    private RandomCharGenerator randomCharGenerator;

    @Autowired
    public Runner(RandomNameGenerator randomNameGenerator) {
        this.randomNameGenerator = randomNameGenerator;
    }

    @Autowired
    public void setRandomCharGenerator(RandomCharGenerator randomCharGenerator) {
        this.randomCharGenerator = randomCharGenerator;
    }
}
```

Dependency Injection – By Qualifier

- Useful if you need to be precise
- This can be an issue in some cases

```
@Component
public class FooService {
    @Autowired
    @Qualifier("fooFormatter")
    private Formatter formatter;
}
```

```
@Component ("fooFormatter")
public class FooFormatter implements Formatter {
    public String format() {
        return "foo";
    }
}
```

```
@Component (“barFormatter")
public class BarFormatter implements Formatter {
    public String format() {
        return "bar";
    }
}
```

Dependency Injection – By Custom Qualifier

- Create custom @Qualifier annotation

```
@Component
public class FooService {
    @Autowired
    @FormatterType("Foo")
    private Formatter formatter;
}
```

```
@Qualifier
@Target({ ElementType.FIELD, ElementType.METHOD,
       ElementType.TYPE, ElementType.PARAMETER})
@Retention(RetentionPolicy.RUNTIME)
public @interface FormatterType {
    String value();
}
```

```
@FormatterType("Foo")
@Component
public class FooFormatter implements Formatter {
    public String format() { return "foo"; }
}
```

```
@FormatterType("Bar")
@Component
public class BarFormatter implements Formatter {
    public String format() { return "bar"; }
}
```

Dependency Injection – By Name

```
@Component ("fooFormatter")
public class FooFormatter implements Formatter {
    public String format() {
        return "foo";
    }
}
```

```
@Component (“barFormatter")
public class BarFormatter implements Formatter {
    public String format() {
        return "bar";
    }
}
```

```
@Component
public class FooService {
    @Autowired
    private Formatter fooFormatter;
}
```

Dependency Injection

- Even if frequently used, « field-based » dependency injection should be avoided
- Eases the violation of the « Single Responsibility Principle »
  - First of five SOLID principles aiming at making software design more
    - Understandable
    - Flexible
    - Maintainable
- Field based injection will keep a code clean even if there is a lot of dependencies

Dependency Injection

UNIVERSITÉ CÔTE D'AZUR

- With constructor-based injection, code will become messy
  - Sending a clear signal something is wrong
  - Having too many collaborators often means the architecture is not well thought
    - Split into smaller pieces
    - Make this part more maintainable
- If you know what you are doing and do not have any immutable need
  - Can use any type of injection

Dependency Injection Design Pattern

- Separates the creation of class dependencies from the class itself
- Transferring this responsibility to a class injector
- Implies a loosely coupled architecture
- Matches the Single Responsibility (1) and Dependency Inversion (5) principles

UNIVERSITÉ
CÔTE D'AZUR

Is it SOLID ?

SOLID Principles

SOLID ?

UNIVERSITÉ CÔTE D'AZUR

- Acronym for 5 design principles aiming at
  - Designing
  - Producing
  - Software architecture
    - Understandable
    - Flexible
    - Maintainable
- Subset of principles promoted by Robert Cecil Martin (famous Software eng.)

SOLID ?

- SOLID principles theory was introduced around 2000
  - Agile Software Development, Principles, Patterns, and Practices (2003, ISBN-13: 978-0135974445)
- Principles
  - Single responsibility principle
  - Open/closed principle
  - Liskov substitution principle
  - Interface segregation principle
  - Dependency inversion principle

Why SOLID ?

UNIVERSITÉ CÔTE D'AZUR

- Development industry is growing fast
- Technologies are evolving
- More and more developer
  - With different coding styles
  - With different skill levels
- Need to define a set of rules to ensure consistent behavior

UNIVERSITÉ
CÔTE D'AZUR

Single Responsibility Principle

SOLID Principles

Single Responsibility Principle

- “A class should have only one reason to change”
  - A “reason to change” should be seen as a responsibility
- Eg: A module in charge of gathering external data and export them
  - The external data link can change
  - The export format can change
  - Two reasons to change, theses changes can be independent
- Single Responsibility Principle suggests the two should be handled separately

Single Responsibility Principle

UNIVERSITÉ CÔTE D'AZUR

- Use one class for one responsibility
  - Class will be more robust
  - Changing one functionality will never break anything but the edited class
- Obvious link with the principle of cohesion
  - Measurement describing the cohesion level inside of a module
  - High cohesion is preferable as it favors
    - Robustness
    - Reliability
    - Reusability

Single Responsibility Principle

UNIVERSITÉ CÔTE D'AZUR

```
public class Cart {

    private Integer id;
    private Float total;
    private List<CartProduct> products = new ArrayList<CartProduct>();
    private Connection connection;

    public void collectTotals() {
        for (CartProduct product: products) {
            total += product.getPrice();
        }
        // [...]
    }

    public void save() throws SQLException {
        this.connection = DriverManager.getConnection("jdbc:mysql://localhost:3306/ecommerce", "root", "");
        Statement stmt = this.connection.createStatement();
        String sql = "insert into cart ...";
        int rs = stmt.executeUpdate(sql);

        // [...]
    }
}
```

Single Responsibility Principle

UNIVERSITÉ CÔTE D'AZUR

- Cart class has two responsibilities (hence reasons to change)
  - *collectTotals()* : Calculate the total price of the cart
    - Taxes rates can change over time
  - *save()* : Persistent method to save the current cart to the database
    - Method is specific to current class, will have to create a new one for each model class
- Not appropriate
  - Robustness
  - Reliability
  - Reusability

Single Responsibility Principle

UNIVERSITÉ CÔTE D'AZUR

- Break responsibilities into several classes
- Two responsibilities → Two classes minimum
- Ideal segmentation level depends on
  - Team skill
  - Project size
  - Context
    - Time constraints
    - Project cost

Single Responsibility Principle

UNIVERSITÉ CÔTE D'AZUR

- Segmentation option could be
  - Externalize and split database connection and persistence order
    - ConnectionDAO : in charge of creating the database connection
    - CartDAO : in charge of the Cart persistence operation
  - Externalize and handle different taxes options
    - TaxRule : Interface to provide unified method to collect totals
    - TaxRate : Enum to handle the different tax rates and provide unified access to each tax rate rule
    - TaxRateRules : Implementation of the TaxRule interface to provide specific tax rate

Single Responsibility Principle

UNIVERSITÉ CÔTE D'AZUR

```
public class ConnectionDAO {

    public Connection doConnect() {
        try {
            // Db type could also be extracted out of here
            setConnection(DriverManager.getConnection(
                    "jdbc:mysql://"
                        + getDbAddress() + ":" + getPort() + "/"
                        + getDbName(), getUsername(), getPassword()));
        } catch (SQLException e) {
            e.printStackTrace();
        }
        return getConnection();
    }
```

Single Responsibility Principle

UNIVERSITÉ CÔTE D'AZUR

```
public class CartDAO {

    public Integer save(Cart cart)
    {
        ConnectionDAO connectionDAO = new ConnectionDAO("username", "password");

        connectionDAO.setDbAddress("localhost");
        connectionDAO.setPort("3306");
        connectionDAO.setDbName("ecommerce");

        Connection connection = connectionDAO.doConnect();

        /* Handle request logic here */

        return cart.getId();
    }
}
```

Single Responsibility Principle

```
public interface TaxRule {

    public Float collectTotals(Cart cart);
}
```

```
public class BasicTaxRateRule implements TaxRule {
    @Override
    public Float collectTotals(Cart cart) {
        return cart.getTotal() * 1.20F;
    }
}
```

Single Responsibility Principle

UNIVERSITÉ CÔTE D'AZUR

```java
public class IntermediateTaxRateRule implements TaxRule {
    @Override
    public Float collectTotals(Cart cart) {
        return cart.getTotal() * 1.10F;
    }
}
```

```java
public class ReducedTaxRateRule implements TaxRule {
    @Override
    public Float collectTotals(Cart cart) {
        return cart.getTotal() * 1.055F;
    }
}
```

Single Responsibility Principle

UNIVERSITÉ CÔTE D'AZUR

```
public enum TaxRate {
    BASIC_TAX_RATE(new BasicTaxRateRule()),
    INTERMEDIATE_TAX_RATE(new IntermediateTaxRateRule()),
    REDUCED_TAX_RATE(new ReducedTaxRateRule());

    private TaxRule taxRule;

    TaxRate(TaxRule taxRule) {
        this.taxRule = taxRule;
    }

    public TaxRule getTaxRule() {
        return taxRule;
    }
}
```

Single Responsibility Principle

UNIVERSITÉ CÔTE D'AZUR

```java
public class Cart {

    private Integer id;
    private Float total;
    private TaxRate taxRate;
    private List<CartProduct> products = new ArrayList<CartProduct>();

    /**
     * If taxes rates are changing, will not have to redo this
     */
    public Float collectTotals() {
        for (CartProduct product: products) {
            total += product.getPrice();
        }
        return taxRate.getTaxRule().collectTotals(this);
    }
```

UNIVERSITÉ
CÔTE D'AZUR

Open/Closed Principle

SOLID Principles

Open/Closed Principle

UNIVERSITÉ CÔTE D'AZUR

- "You should be able to extend a class behavior without modifying it"
- Open to extension
- Closed to modification
- Extend behavior through
  - Inheritance
  - Interface
  - Composition

Open/Closed Principle

UNIVERSITÉ CÔTE D'AZUR

```java
public class CartProduct {

    private Double price;
    private Integer quantity;

    // Set the current cart line price considering the client type and the shipping options
    public Double getLinePrice()
    {
        Double rawPrice = this.price * this.quantity;
        Double discount;
        ClientType clientType = UserService.getCurrentUserType();
        if (clientType ==  ClientType.CLIENT_CLASSIC )
            discount = new ClientDiscount().getDiscount(rawPrice);
        else if  (clientType ==  ClientType.CLIENT_PRO )
            discount = new ProDiscount().getDiscount(rawPrice);
        else discount = 1.0;
        /* Other rules changing the price could also be added */
        rawPrice = discount * rawPrice;
        return rawPrice;
    }
}
```

Open/Closed Principle

UNIVERSITÉ CÔTE D'AZUR

```java
public class ClientDiscount {

    public Double getDiscount(Double price)
    {
        if (price > 1000)
            return 0.8;
        else if (price > 500)
            return 0.9;
        else if (price > 100)
            return 0.95;
        else return 1.0;
    }
}
```

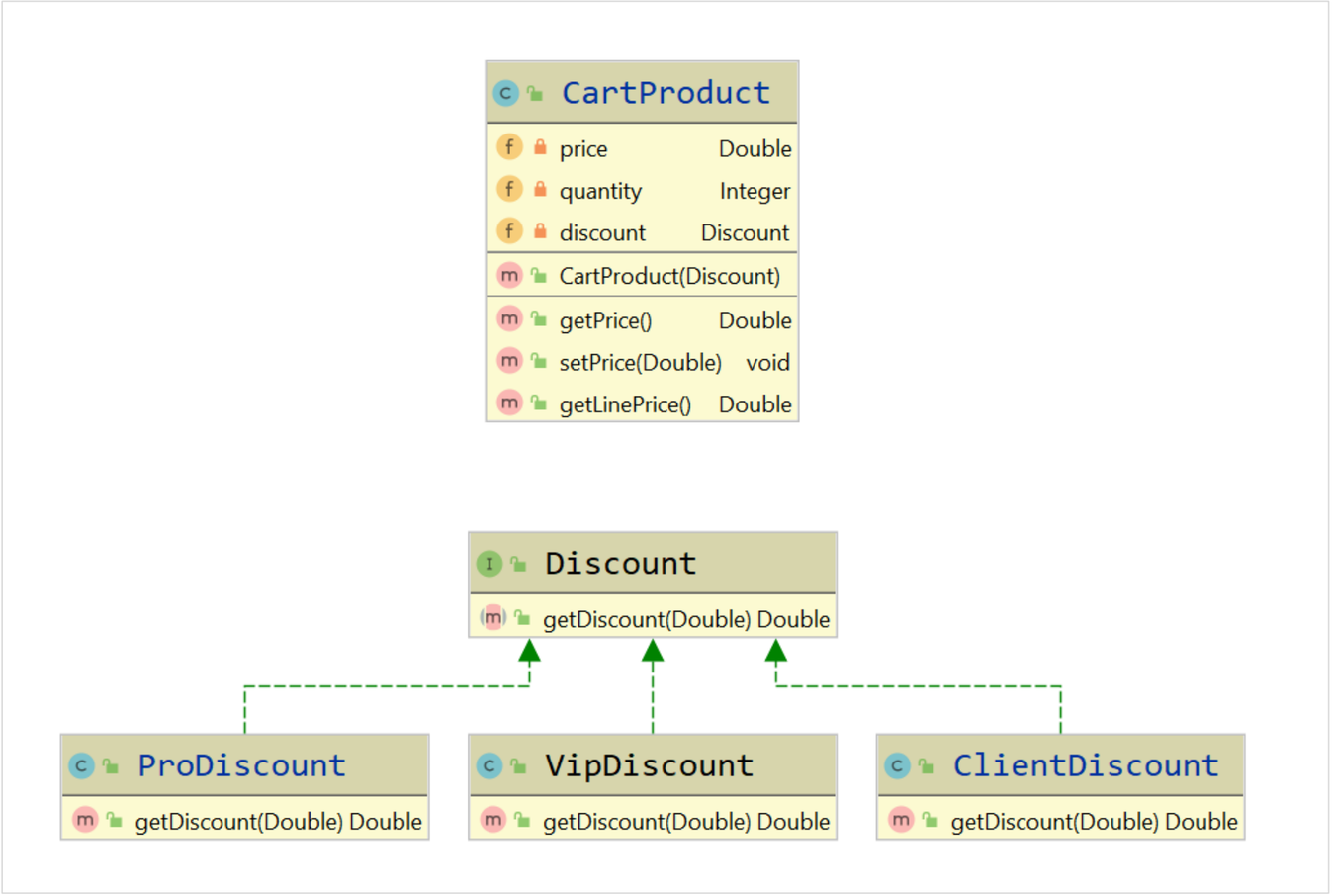
```java
public class ProDiscount {

    public Double getDiscount(Double price)
    {
        if (price > 10000)
            return 0.60;
        else if (price > 5000)
            return 0.70;
        else if (price > 1000)
            return 0.80;
        else return 1.0;
    }
}
```

Open/Closed Principle

UNIVERSITÉ CÔTE D'AZUR

- Multiple issues
  - Extending this behavior to new “Client types” will force the edit of “CartProduct” class
  - High complexity
  - Adding new rules to calculate the price will increase the overall complexity
- Solution ?
  - Create an interface to handle the discount calculation task
  - Adding a new “Client type” will not force any edit on the other files

Open/Closed Principle



SOLID

Open/Closed Principle

UNIVERSITÉ CÔTE D'AZUR

```java
public interface Discount {

    public Double getDiscount(Double price);
}
```

```java
public class ProDiscount implements Discount {
    @Override
    public Double getDiscount(Double price)
    {
       /* Logic here */
    }
}
```

```java
public class ClientDiscount implements Discount {

    @Override
    public Double getDiscount(Double price)
    {
       /* Logic here */
    }
}
```

```java
public class VipDiscount implements Discount {
    @Override
    public Double getDiscount(Double price)
    {
       /* Logic here */
    }
}
```

Open/Closed Principle – before

UNIVERSITÉ CÔTE D'AZUR

```java
public class CartProduct {

    private Double price;
    private Integer quantity;

    // Set the current cart line price considering the client type and the shipping options
    public Double getLinePrice()
    {
        Double rawPrice = this.price * this.quantity;
        Double discount;
        ClientType clientType = UserService.getCurrentUserType();
        if (clientType ==  ClientType.CLIENT_CLASSIC )
            discount = new ClientDiscount().getDiscount(rawPrice);
        else if  (clientType ==  ClientType.CLIENT_PRO )
            discount = new ProDiscount().getDiscount(rawPrice);
        else discount = 1.0;
        /* Other rules changing the price could also be added */
        rawPrice = discount * rawPrice;
        return rawPrice;
    }
}
```

Open/Closed Principle - after

UNIVERSITÉ CÔTE D'AZUR

```java
public class CartProduct {

    private Double price;
    private Integer quantity;
    private Discount discount;

    public CartProduct(Discount discount) {
        this.discount = discount;
    }

    /* Price handling depending on client type not handled here anymore */
    public Double getLinePrice()
    {
        Double rawPrice = this.price * this.quantity;
        Double discount = this.discount.getDiscount(rawPrice);
        rawPrice = discount * rawPrice;

        return rawPrice;
    }
}
```

Open/Closed Principle

- With this design
  - Overall complexity of the “CartProduct” class is greatly reduced
  - Adding new rules will not be a pain if following the same pattern (shipping cost)
  - Adding new “ClientType” by creating a new class implementing the “Discount” interface
- Now able to extend (“Discount” interface) the behavior (of “CartProduct”) without modifying it

UNIVERSITÉ
CÔTE D'AZUR

Liskov Substitution Principle

SOLID Principles

Liskov Substitution Principle

UNIVERSITÉ CÔTE D'AZUR

- "Subtype Requirement: Let ϕ (x) be a property provable about objects x of type T. Then ϕ (y) should be true for objects y of type S where S is a subtype of T."
- "If S is a declared subtype of T, objects of type S should behave as objects of type T are expected to behave, if they are treated as objects of type T"
- "Derived classes must be substitutable for their base classes"
- "Superclasses should be replaceable with objects of a subclass without altering the correctness of the program "

Liskov Substitution Principle

UNIVERSITÉ CÔTE D'AZUR

- Reminder : In a Java environment, S is a declared subtype of T if
  - S is T
  - S implements or extends T
  - S implements or extends a type that implements or extends T …

Liskov Substitution Principle

UNIVERSITÉ CÔTE D'AZUR

- Let's demonstrate with examples

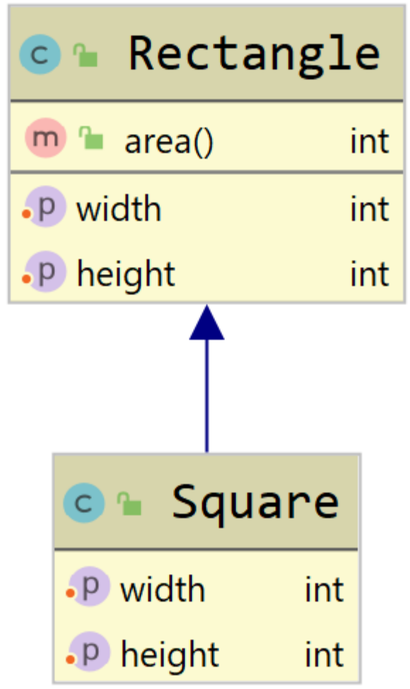
```
public class Rectangle {

    private int width;
    private int height;

    public void setWidth(int width) { this.width = width; }

    public void setHeight(int height) { this.height = height; }

    public int area()
    {
        return this.height * this .width;
    }
}
```

Liskov Substitution Principle

UNIVERSITÉ CÔTE D'AZUR

- Now let's implement a Square class
  - We will extend Rectangle as it is almost the same

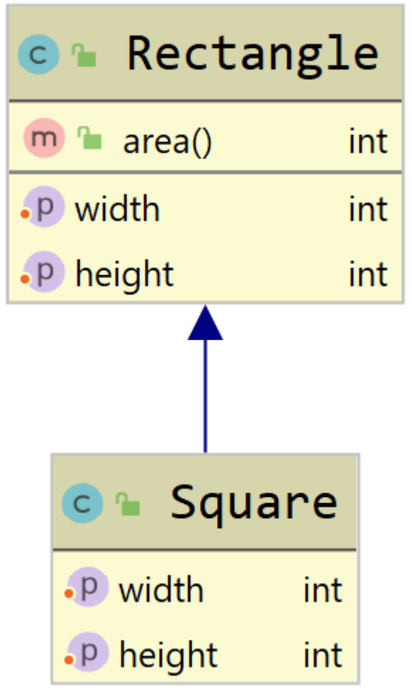
```
public class Square extends Rectangle {
    @Override
    public void setWidth(int width) {
        super.setWidth(width);
        super.setHeight(width);
    }
    @Override
    public void setHeight(int height) {
        super.setWidth(height);
        super.setHeight(height);
    }
}
```



Liskov Substitution Principle

UNIVERSITÉ CÔTE D'AZUR

- We can use Square wherever Rectangle is expected because
  - Square is extending Rectangle
  - Squares and Rectangles behave the same way
- Except …

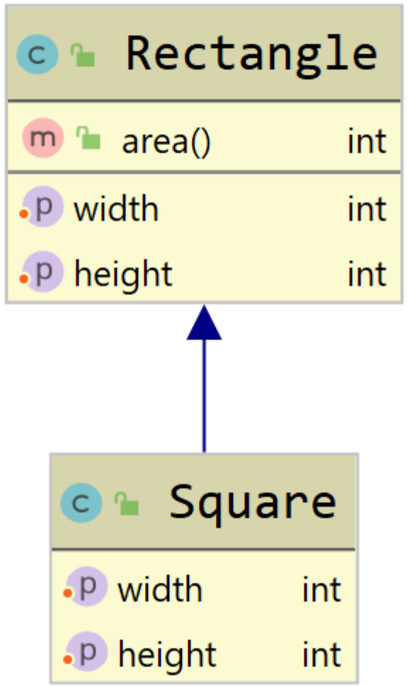


Liskov Substitution Principle

UNIVERSITÉ CÔTE D'AZUR

- … this will break when using Square instead of Rectangle
- But Square is a subtype of Rectangle
- I should be able to do this with proper inheritance

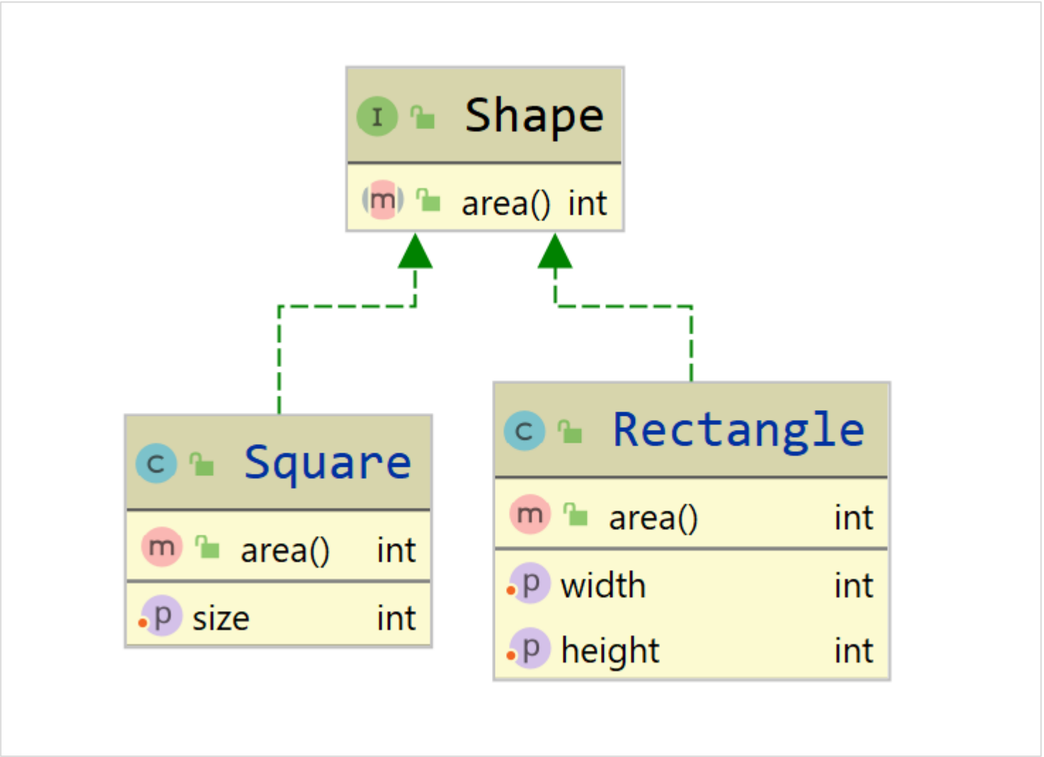
```
public class SomeClass {
    public void someMethod(Rectangle rectangle)
    {
        rectangle.setWidth(2);
        rectangle.setHeight(3);
        assert rectangle.area() == 6;
    }
}
```



Liskov Substitution Principle

UNIVERSITÉ CÔTE D'AZUR

- According to Liskov Substitution principle, a valid solution would be



Liskov Substitution Principle

UNIVERSITÉ CÔTE D'AZUR

- With this kind of implementation

```java
public interface Shape {
    public int area();
}
```

```java
public class Rectangle implements Shape {

    public int width;
    public int height;

    public void setWidth(int width) { this.width = width; }

    public void setHeight(int height) { this.height = height; }

    @Override
    public int area()
    {
        return this.height * this.width;
    }
}
```

```java
public class Square implements Shape {

    public int size;

    public void setSize(int size)
    {
        this.size = size;
    }

    @Override
    public int area()
    {
        return this.size * this.size;
    }
}
```

Liskov Substitution Principle

UNIVERSITÉ CÔTE D'AZUR

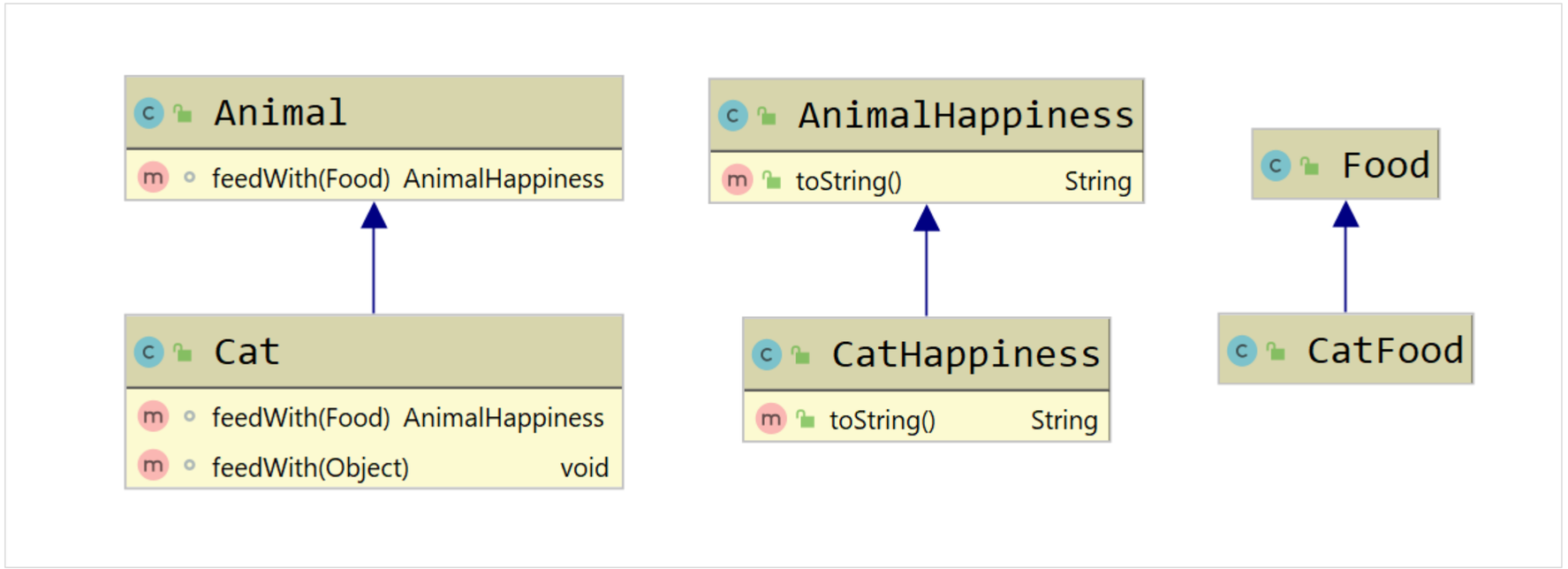
- LSP also declares that "declared subtypes should be **behavioral subtypes**"
  - Adding the requirement for behavioral substitutability
- Tight link with contract programming, defining that
  - Preconditions cannot be reinforced in the subtypes
    - Cannot have a subclass requiring stronger preconditions than the superclass
  - Postconditions cannot be weakened in the subtypes
    - Cannot have a subclass with postconditions weaker than the superclass's

Liskov Substitution Principle

UNIVERSITÉ CÔTE D'AZUR

- Constraints on signatures
  - Contravariance of method arguments in the subtype (not possible with Java)
  - Covariance of return types in the subtype
  - No new exceptions should be thrown by methods of the subtype
    - Except when exceptions are subtypes of exceptions thrown by the methods of the supertype

Liskov Substitution Principle



Liskov Substitution Principle

UNIVERSITÉ CÔTE D'AZUR

```java
public class Animal {
    AnimalHappiness feedWith(Food food)
    { return new AnimalHappiness(); }
}
```

```java
public class Cat extends Animal
{
    // Covariance of return type
    // Contravariance of method argument
    // /!\ Do not feed with CatFood /!\
    @Override
    AnimalHappiness feedWith(Food food)
    { return new CatHappiness(); }

    // Should demonstrate contravariance argument
    // But Java doesn't allow it
    // Considered it as an overloading
    void feedWith(Object food)
    { /* ... */ }
}
```

Liskov Substitution Principle

UNIVERSITÉ CÔTE D'AZUR

```java
public class AnimalHappiness {
    @Override
    public String toString() {
        return "Animal is making happy noises !";
    }
}
```

```java
public class Food {
}
```

```java
public class CatHappiness extends AnimalHappiness {
    @Override
    public String toString() {
        return "The cat if purring !";
    }
}
```

```java
public class CatFood extends Food {
}
```

Liskov Substitution Principle

UNIVERSITÉ CÔTE D'AZUR

- Quick way to detect a LSP violation
  - If you are using any kind of "instanceof" operator
  - Your inheritance definition is wrong

```
public class SomeClass {
    public void someMethod(Object rectangle)
    {
        rectangle.setWidth(2);
        rectangle.setHeight(3);
        if (rectangle instanceof Rectangle)
            assert rectangle.area() == 6;
        else if (rectangle instanceof Square)
            assert rectangle.area() == 9;
    }
}
```

UNIVERSITÉ
CÔTE D'AZUR

Interface Segregation Principle

SOLID Principles

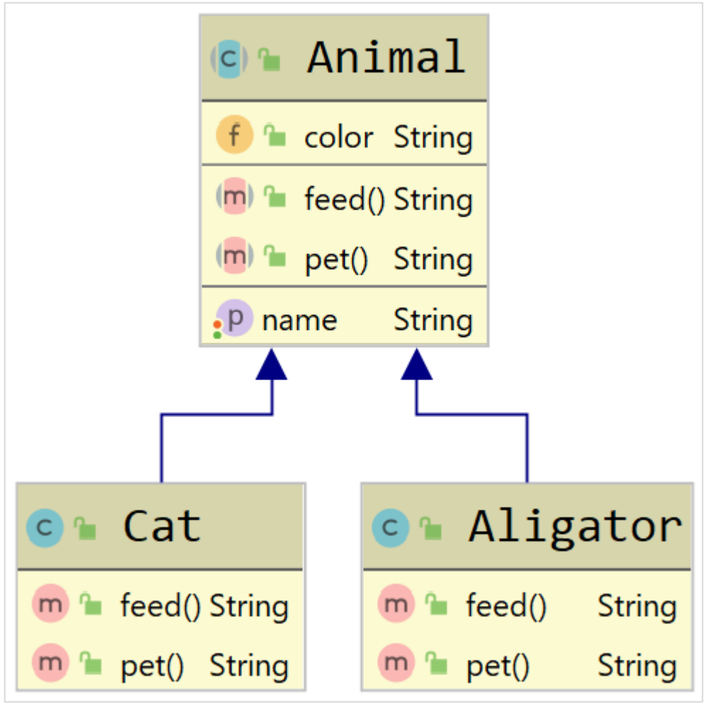
Interface Segregation Principle

UNIVERSITÉ CÔTE D'AZUR

- "*Make fine grained interfaces that are client specific* "
- Many specialized interfaces are better than one general interface
  - Maintain a good cohesion within your interfaces
  - Specialized interfaces are easier to maintain and evolve

Interface Segregation Principle

- Global idea seams about right
- Animal Abstract class
  - Defines color and name properties
  - Declares feed and pet methods



Interface Segregation Principle

UNIVERSITÉ CÔTE D'AZUR

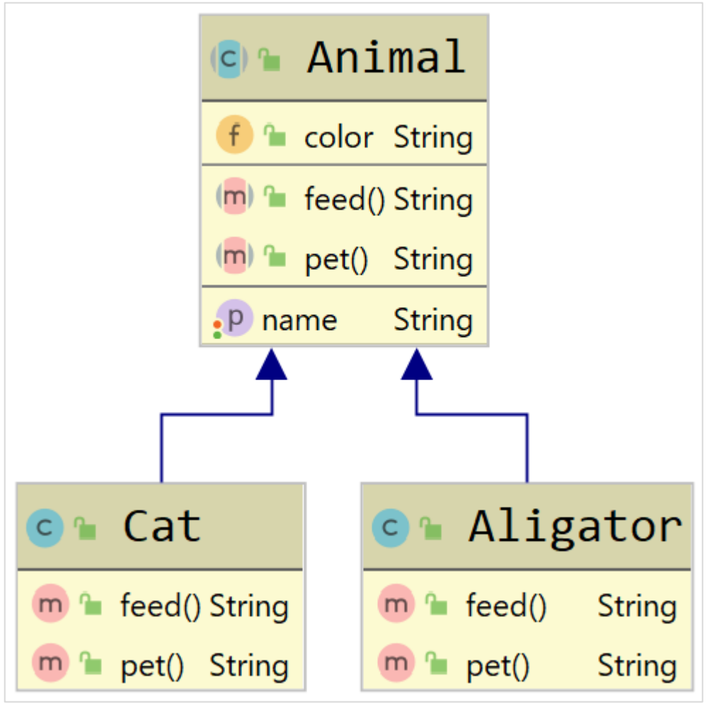
```java
public abstract class Animal {

    public String color;
    public String name;

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public abstract String feed();

    public abstract String pet() throws Exception;
}
```

```java
public class Cat extends Animal {
    @Override
    public String feed() {
        return "Garfield is happy !";
    }
    @Override
    public String pet() {
        return "Garfield is purring";
    }
}
```

```java
public class Aligator extends Animal {
    @Override
    public String feed() {
        return "Hector is happy !";
    }
    @Override
    public String pet() throws Exception {
        throw new Exception("Never do that you fool !");
    }
}
```

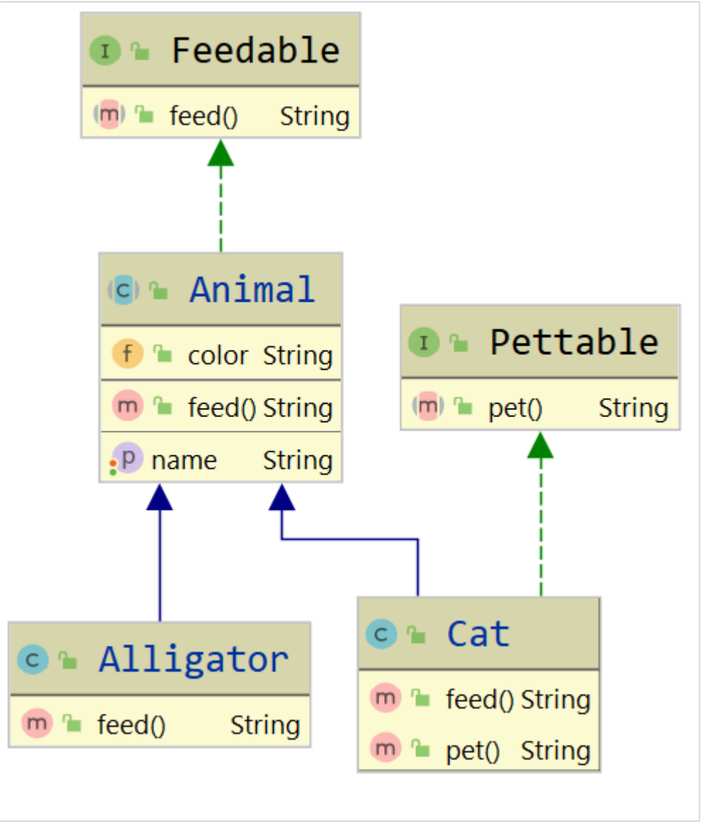
Interface Segregation Principle

- Base model seams right
- However, petting an Alligator doesn’t feel right !
- How can we do this better ?
  - Refactor the code to break these aspects into two interfaces
    - Feedable
    - Pettable



Interface Segregation Principle

- Animal implements Feedable
- Pettable interface dedicated to “Domestic animals”
- Alligator is just a common “Animal”
  - Could remove it



Interface Segregation Principle

UNIVERSITÉ CÔTE D'AZUR

```java
public interface Feedable {
    public String feed();
}
```

```java
public interface Pettable {
    public String pet();
}
```

```java
public class Cat extends Animal implements Pettable {
    @Override
    public String feed() {
        return "Garfield is happy !";
    }
    @Override
    public String pet() {
        return "Garfield is purring";
    }
}
```

```java
public class Alligator extends Animal {
    @Override
    public String feed() {
        return "Hector is happy !";
    }
}
```

```java
public abstract class Animal implements Feedable  {

    public String color;
    public String name;

    public String getName() {
        return name;
    }

    @Override
    public String feed()
    {
        return "Animals are happy when being fed !";
    }

    public void setName(String name) {
        this.name = name;
    }
}
```

MIAGE RÉSEAU DES MIAGE DE FRANCE Nice

SOLID

Interface Segregation Principle

- Helps us respect the other principles
- Helps reducing “bad” coupling
- Do not overdo it
  - Do not create interfaces just for fun
  - Should always aim at making more sense

UNIVERSITÉ
CÔTE D'AZUR

Dependency Inversion Principle

SOLID Principles

Dependency Inversion Principle

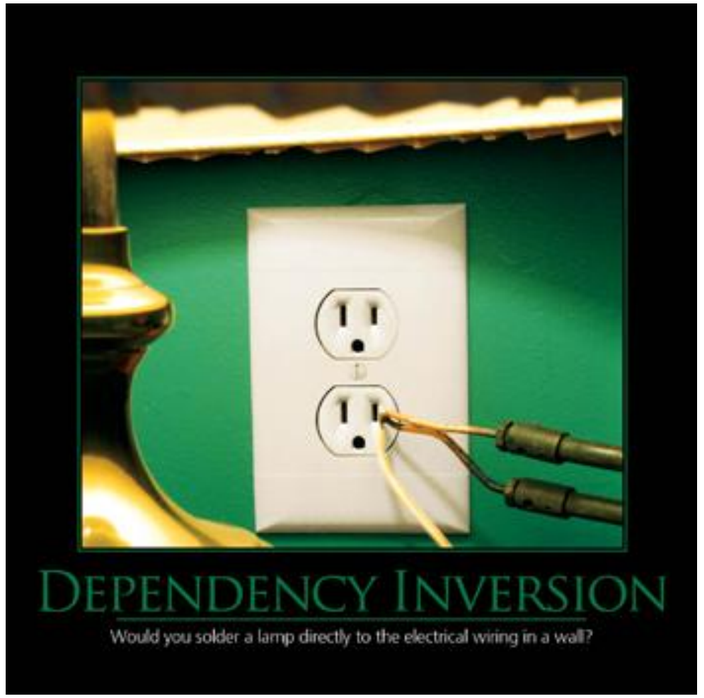
UNIVERSITÉ CÔTE D'AZUR

- *"Depend on abstractions, not on concretions"*
  - *"High-level modules should not depend on low-level modules. Both should depend on abstractions"*
  - *"Abstractions should not depend on details. Details (concrete implementations) should depend on abstractions"*
- Reversing the classic way of thinking with object oriented programming
  - High & low level must depend on the same abstraction

Dependency Inversion Principle

UNIVERSITÉ CÔTE D'AZUR

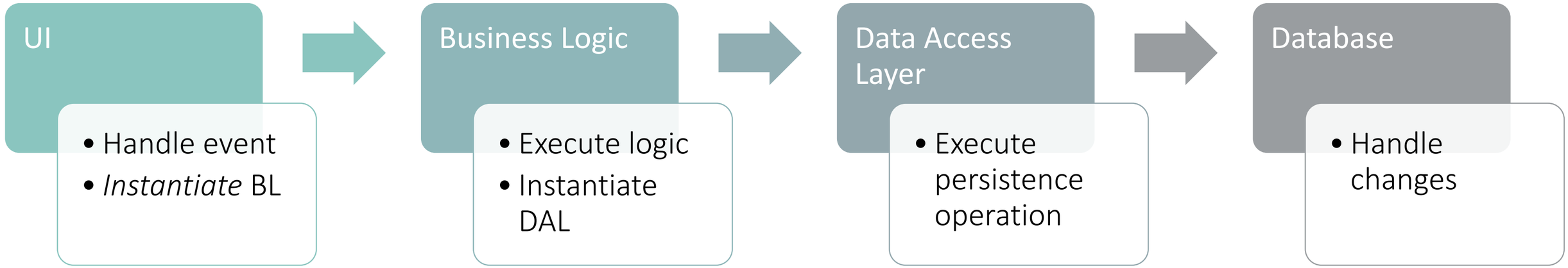
- When applying OCP & LSP
  - Dependency inversion principle will be a natural consequence
- Objectives
  - Reduce the system coupling
  - Improves code reusability
  - Improves code testability



Dependency Inversion Principle

UNIVERSITÉ CÔTE D'AZUR

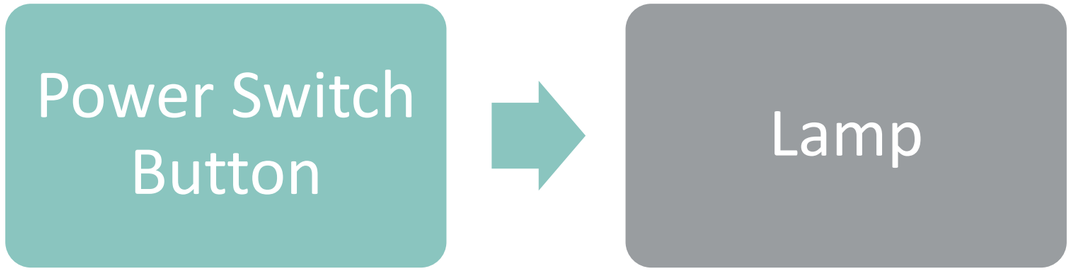
- Classic schema
  - Tight coupling
  - Transitive dependencies towards DB



Dependency Inversion Principle

UNIVERSITÉ CÔTE D'AZUR

- Let's have a simpler and more concrete example to work with !



Dependency Inversion Principle

UNIVERSITÉ CÔTE D'AZUR

```
public class PowerSwitchButton {
    public Lamp livingRoomLamp;
    public PowerSwitchButton(Lamp livingRoomLamp) {
        this.livingRoomLamp = livingRoomLamp;
    }
    public void closeCircuit() {
        livingRoomLamp.turnOn();
    }
    public void openCircuit() {
        livingRoomLamp.turnOff();
    }
}
```

```
public class Lamp {

    public String turnOn() {
        return "Light is on !";
    }

    public String turnOff() {
        return "Light is off !";
    }

}
```


MIAGE RÉSEAU DES MIAGE DE FRANCE Nice

SOLID

Dependency Inversion Principle

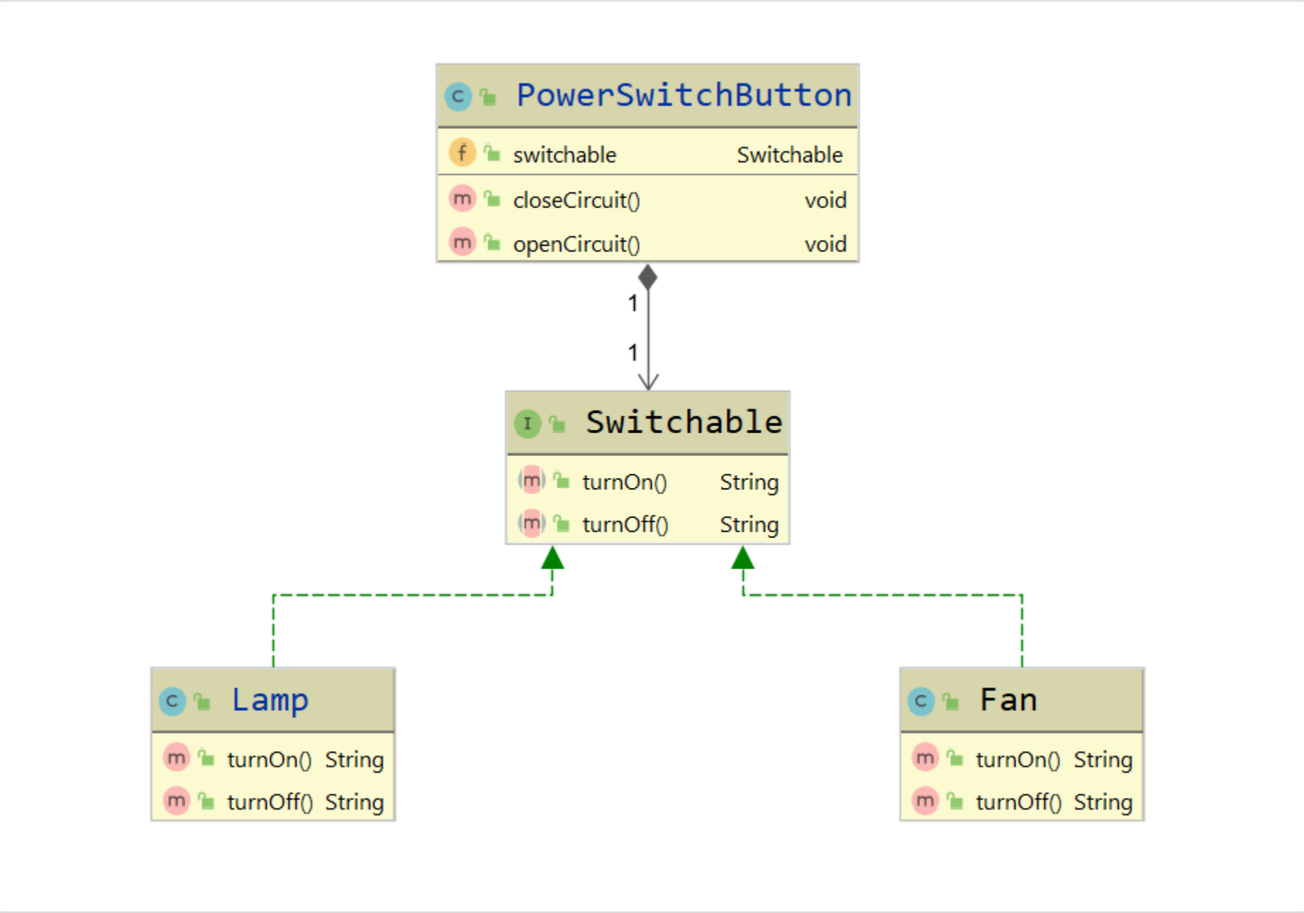
UNIVERSITÉ CÔTE D'AZUR

- Issues
  - High level class depends on lower-level class
    - Reference in the PowerSwitchButton class towards Lamp class
  - Lamp reference is hardcoded in the PowerSwitchButton class
    - We cannot plug anything else but a Lamp to this switch ?!
  - Each time we want to add a new “thing” the switch class will have to change
- Everything is wrong
  - Apply dependency inversion principle to change this

Dependency Inversion Principle

- *"High-level modules should not depend on low-level modules. Both should depend on abstractions"*
  - Lamp & PowerButtonSwitch will have to depend on a new abstraction
- Switchable interface will handle the "Switchable" aspect
  - PowerButtonSwitch will have a Switchable property
  - Lamp will implement Switchable interface and override to define specific behavior
  - We will then be able to plug something else on that switch !

Dependency Inversion Principle



Dependency Inversion Principle

UNIVERSITÉ CÔTE D'AZUR

```
public class PowerSwitchButton {
    public Switchable switchable;

    public PowerSwitchButton(Switchable switchable) {
        this.switchable = switchable;
    }

    public void closeCircuit() {
        switchable.turnOn();
    }
    public void openCircuit() {
        switchable.turnOff();
    }
}
```

```
public interface Switchable {
    public String turnOn();
    public String turnOff();
}
```

Dependency Inversion Principle

UNIVERSITÉ CÔTE D'AZUR

```java
public interface Switchable {
    public String turnOn();
    public String turnOff();
}
```

```java
public class Lamp implements Switchable  {
    @Override
    public String turnOn() {
        return "Light is on !";
    }
    @Override
    public String turnOff() {
        return "Light is off !";
    }
}
```

```java
public class Fan implements Switchable  {
    @Override
    public String turnOn() {
        return "Fan is on !";
    }
    @Override
    public String turnOff() {
        return "Fan is off !";
    }
}
```

Dependency Inversion Principle

UNIVERSITÉ CÔTE D'AZUR

- Enable easy testing !

```
public class SwitchTest {
    @Test
    public void testToggleSwitch()
    {
        Switchable switchableLamp = new Lamp();
        PowerSwitchButton lampButton = new PowerSwitchButton(switchableLamp);
        lampButton.closeCircuit();
        lampButton.openCircuit();

        Switchable switchableFan = new Fan();
        PowerSwitchButton fanButton = new PowerSwitchButton(switchableFan);
        fanButton.closeCircuit();
        fanButton.openCircuit();
    }
}
```

Dependency Inversion Principle

UNIVERSITÉ CÔTE D'AZUR

- What's the point ? How is it related to Spring ?
  - Remember Dependency Injection
- We said "Dependency Injection" helps to be compliant with
  - Single Responsibility Principle (1)
  - Dependency Inversion Principle (5)
- Dependency **Inversion** Principle define a code structure to achieve loosely coupled modules
- Dependency **Injection** is a pattern defining the wiring of your dependencies

UNIVERSITÉ
CÔTE D'AZUR

IoC, DI, SRP, OCP, LSP, ISP, DIP ?!?

Lets clarify !

Acronyms

- SOLID Principles
  - SRP : Single Responsibility Principle
  - OCP : Open / Closed Principle
  - LSP : Liskov Substitution Principle
  - ISP : Interface Segregation Principle
  - DIP : Dependency Inversion Principle
- IoC : Inversion of Control
- DI : Dependency Injection Pattern

Principles

- These are principles
  - SOLID Principles
  - SRP : Single Responsibility Principle
  - OCP : Open / Closed Principle
  - LSP : Liskov Substitution Principle
  - ISP : Interface Segregation Principle
  - DIP : Dependency Inversion Principle

Principles

- High level best practices guidelines
- No specific implementation details
- Not bound to any programming language
- Do not use without a good reason
  - Must match the company strategy
  - Must be shared with all the development team

Design Pattern

- Solution for a commonly occurring issue
- Description on how to tackle and solve an issue
- Set of best practices
- Helps designing a better system
- Design Patterns: Elements of Reusable Object-Oriented Software
  - 1994 – Gang of Four aka “GoF”
  - Description of 23 classic software design patterns

Design Pattern

- Split into 3 families (according to GOF)
  - Creational
    - Factory
    - Prototype
    - Singleton
  - Structural
    - Decorators
    - Proxy
  - Behavioral
    - Iterator
    - Observer

UNIVERSITÉ
CÔTE D'AZUR

IoC Principle

IoC Principle

- **Architectural** design pattern / design principle to achieve **loose coupling** between classes
- Execution flow is not controlled by the application but by the framework
- IoC is essential if working with a Test-driven development process
  - TDD is not possible without IoC
- Often used in conjunction with DIP to achieve **loose coupling**

IoC Principle / DI / IoC / IoC Container

- **Dependency Injection** (pattern) is one of many implementation of IoC (principle)

- **IoC Container** is a framework that implements IoC principle through automatic dependency injection

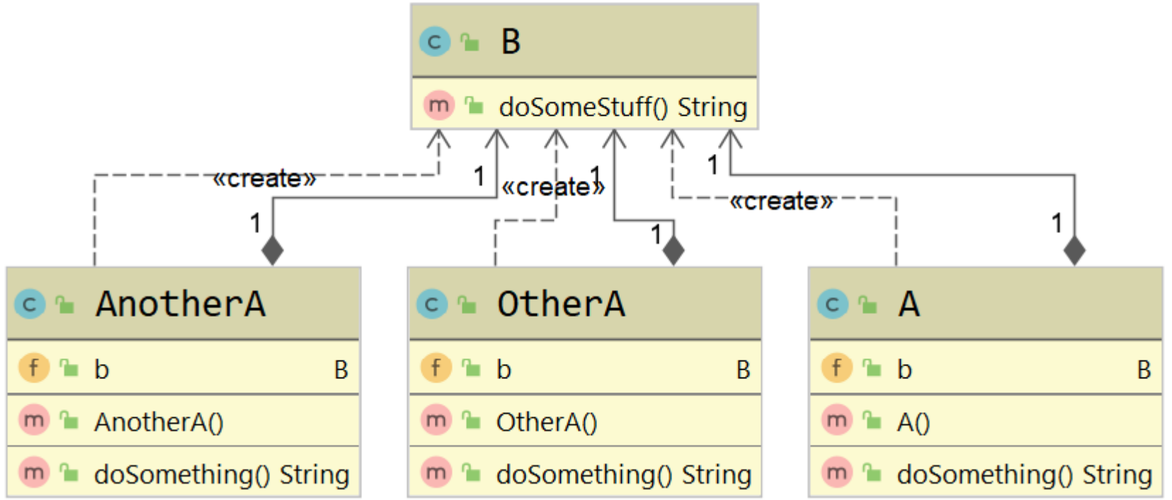
What’s the issue ?

```
public class A {
    public B b;

    public A() {
        this.b = new B();
    }

    public String doSomething()
    {
        return b.doSomeStuff();
    }
}
```

```
public class B {

    public String doSomeStuff()
    {
        return "Doing some stuff !";
    }
}
```

What's the issue ?

- If many classes like A need to use B
  - A design change in B will force a change on A and all other classes using B
- You cannot test A on its own (TDD) as you need B when creating A



What’s the issue ?

- A need B to complete its task
- A is dependent on the B class
- B class is a dependency of A
- To achieve loose coupling
  - Must extract the creation and management of B class from the class A
  - Delegate this to another class : Factory to the rescue !

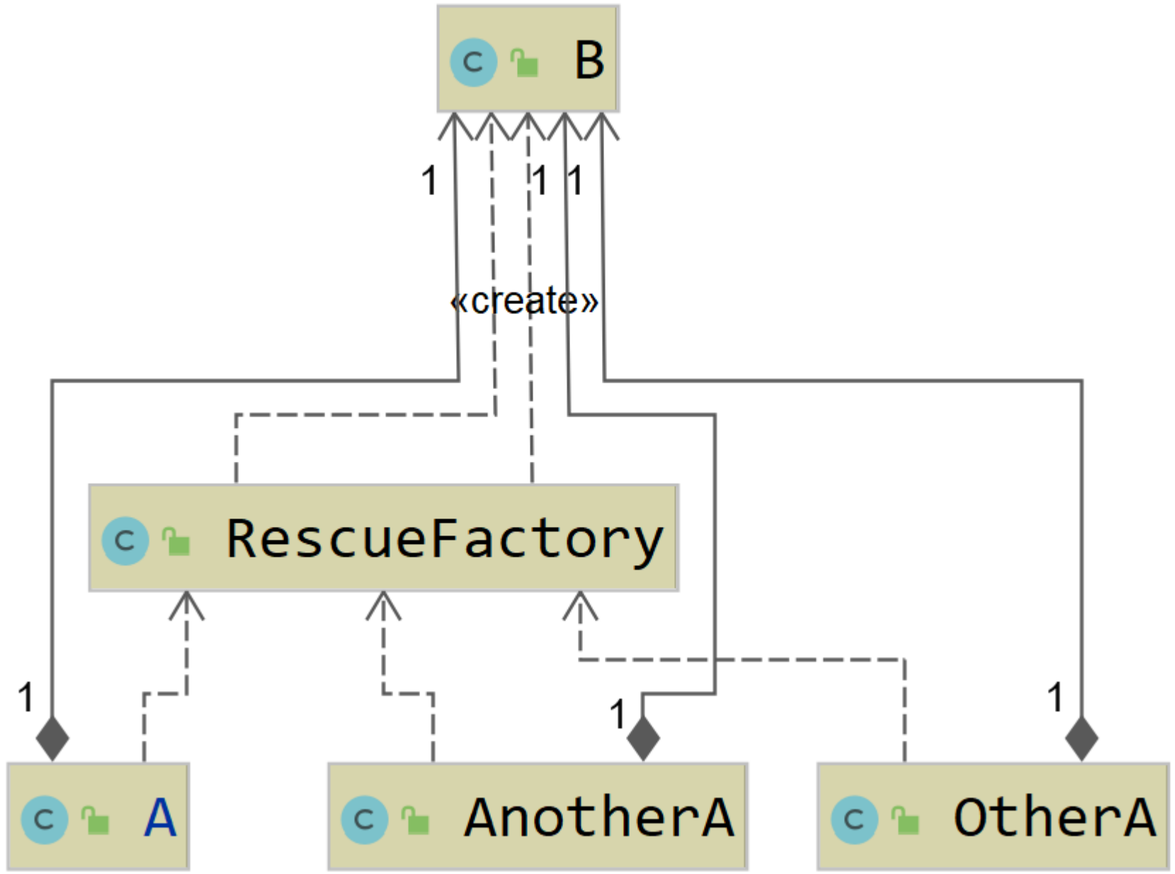
What’s the issue ?

- Extracted the dependency from A to RescueFactory

```
public class RescueFactory {
    public static B getBInstance()
    {
        return new B();
    }
}
```

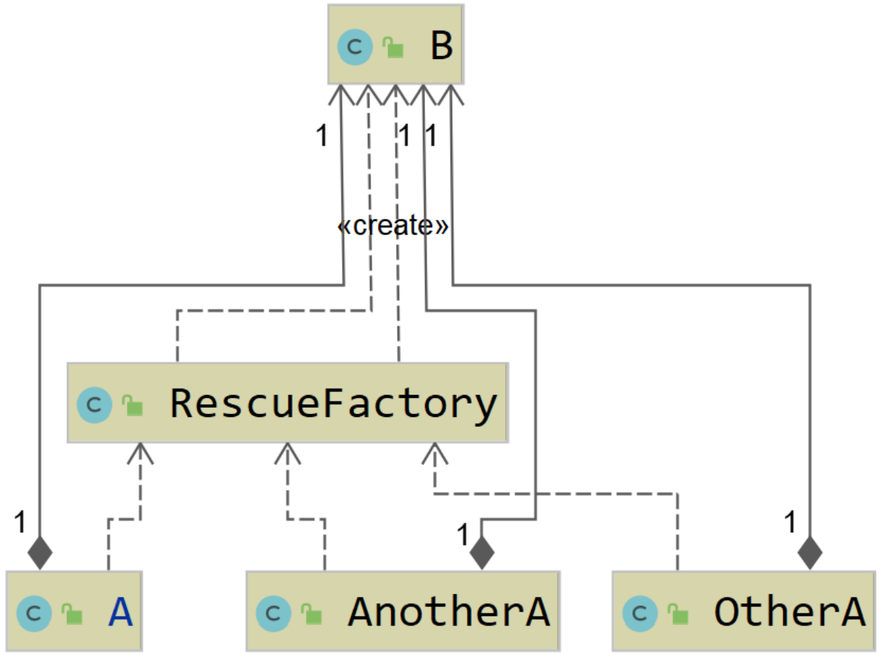
```
public class A {
    public B b;

    public A() {
        this.b = RescueFactory.getBInstance();
    }

    public String doSomething()
    {
        return b.doSomeStuff();
    }
}
```

What’s the issue ?



What's the issue ?

- Solution to one issue
  - A change in the B class will imply a change only in the Factory instead of 3 classes
- Control is inverted
  - From A to Factory
- Coupling is still not loose
- Cannot test A on its own still
- Need DIP to push this approach



UNIVERSITÉ
CÔTE D'AZUR

IoC and Dependency Inversion Principle

Follow DIP to obtain loose coupling

IoC Principle featuring DIP

- More concrete example

```
public class User {

    public String getUserName(int id)
    {
        UserDAL userDAL = UserFactory.getUserDAL();

        return userDAL.getUserName(id);
    }
}
```

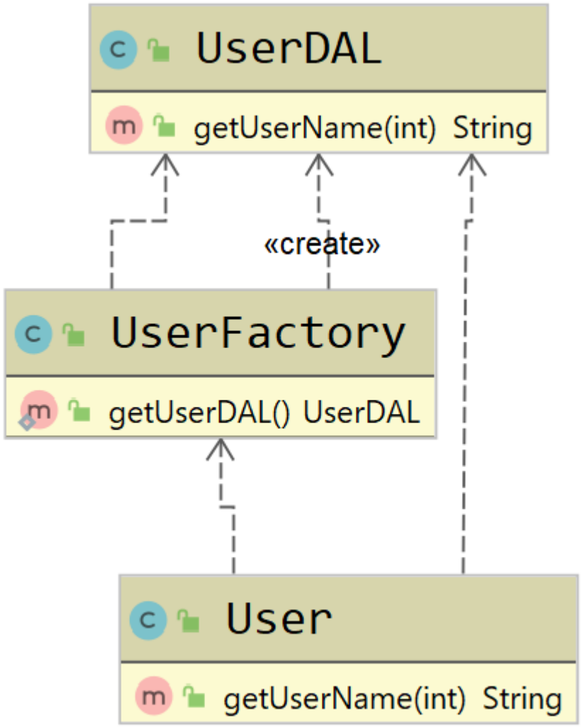
```
public class UserFactory {

    public static UserDAL getUserDAL()
    {
        return new UserDAL();
    }
}
```

```
public class UserDAL {

    public String getUserName(int id)
    {
        return "Retrieve User name from id";
    }
}
```

- Need to get rid of the tight coupling
  - DIP : High & low level must depend on the same abstraction

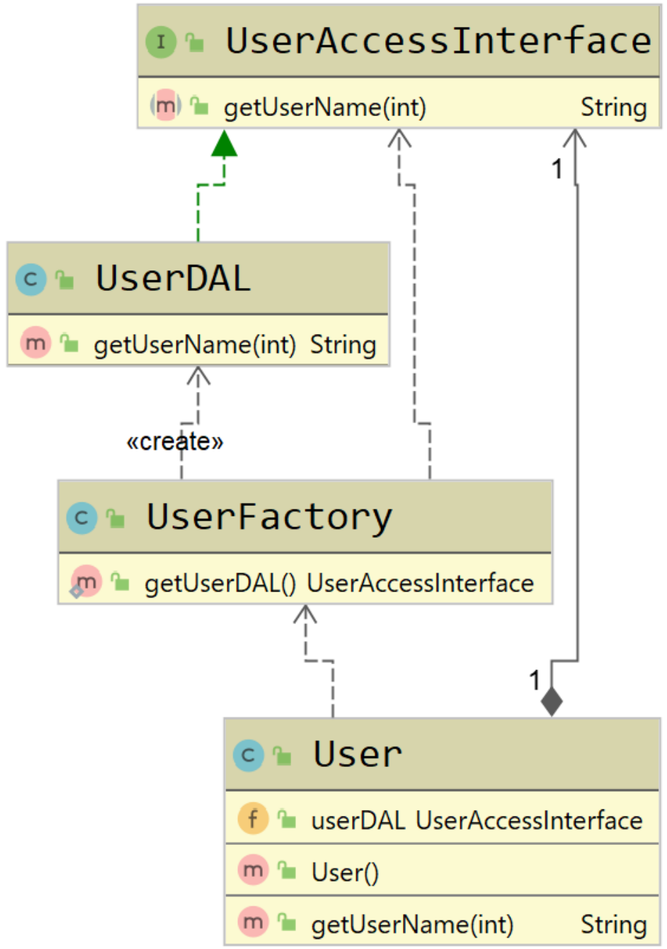
IoC Principle featuring DIP

- User declares no **`new UserDAL()`** but still uses it
- Control is inverted but coupling is remaining
- Following Dependency Inversion Principle
  - UserDAL and User should rely on abstraction
  - Solution : creating an interface to handle this
  - Factory should then handle User the interface



IoC Principle featuring DIP

- Interface created
  - Implemented by UserDAL
- UserFactory now delivers UserAccessInterface
- User using said interface to retrieve data
- UserDAL & User are relying on abstraction
  - UserDAL & User are loosely coupled
  - We can replace UserDAL without impacting User



IoC Principle featuring DIP

- Still tight coupling
  - User class still depends on UserFactory implementation
  - Need to setup some sort of contract between the two
- Lets fix this
  - First let’s make UserFactory a Singleton
    - Do not have to create a new instance each time

IoC Principle featuring DIP

UNIVERSITÉ CÔTE D'AZUR

```
public final class UserFactory {

    private static final UserFactory INSTANCE = new UserFactory();

    public UserFactory() { }

    public static UserFactory getInstance() {
        return UserFactory.INSTANCE;
    }

    public static UserAccessInterface getUserDAL()
    {
        return new UserDAL();
    }
}
```

IoC Principle featuring DIP

- And now let’s use this singleton in the User class

```
public class User {

    private UserFactory userFactory = UserFactory.getInstance();

    public UserAccessInterface userDAL;

    public User() {
        this.userDAL = userFactory.getUserDAL();
    }

    public String getUserName(int id)
    {
        return userDAL.getUserName(id);
    }
}
```

- That’s better but we could use more abstraction

IoC Principle featuring DIP

- At this step
  - User class still depends on the UserFactory implementation
  - Violates the DIP
- Need to implement something that will inject the UserFactory
  - Solution is to set up a dependency injection to get rid of this
- Let’s go old school !

UNIVERSITÉ
CÔTE D'AZUR

IoC & DIP & Dependency Injection

Implement DI to obtain loose coupling

IoC & DIP & DI

- We are going to need a bit of refactoring
  - Not going to use Spring DI functionnalities
- Steps to follow
  - Create an new abstraction layer between UserFactory and User class (1)
    - Get rid of this UserFactory recuperation into the User class
    - Refactor User class to extend this abstraction
  - Create a listener that will be initialized with the application start (2)
    - Inject needed instances into our new abstraction layer

IoC & DIP & DI

```
public abstract class AbstractResource {

    private static UserFactory userFactory;

    protected static UserFactory getUserFactory() {
        return userFactory;
    }

    public static void setUserFactory(UserFactory userFactory) {
        AbstractResource.userFactory = userFactory;
    }
}
```

IoC & DIP & DI

```
public class User {

    public UserAccessInterface userDAL;

    public User() {
        this.userDAL = UserFactory.getUserDAL();
    }

    public String getUserName(int id)
    {
        return userDAL.getUserName(id);
    }
}
```

```
public class User extends AbstractResource {

    public String getUserName(int id) {
        return getUserFactory()
                    .getUserDAL()
                    .getUserName(id);
    }
}
```

IoC & DIP & DI

- Listener creation
  - Create a UserFactory instance
  - Provide this instance to AbstractResource

```
public class DependencyInjectionListener implements ServletContextListener{
    @Override
    public void contextInitialized(ServletContextEvent sce) {

        UserFactory userFactory = new UserFactory();

        AbstractResource.setUserFactory(userFactory);
    }
}
```

IoC & DIP & DI

UNIVERSITÉ CÔTE D'AZUR

- Register the listener

```
@SpringBootApplication
public class SpringbootApplication  {

	@Bean
	public ServletListenerRegistrationBean<ServletContextListener> listenerRegistrationBean() {
	    ServletListenerRegistrationBean<ServletContextListener> bean =
	            new ServletListenerRegistrationBean<>();
	    bean.setListener(new DependencyInjectionListener());
	    return bean;
	}

}
```

IoC & DIP & DI

- Last issues remaining
  - UserFactory is creating a new instance of UserDAL after each call to getUserDAL
  - UserFactory is still a Singleton, this is an issue from a testing point of view
- Solution to these issues ?
  - Use Dependency Injection again !

IoC & DIP & DI

```
public final class UserFactory {

    private static final UserFactory INSTANCE
              = new UserFactory();

    public UserFactory() { }

    public static UserFactory getInstance() {
        return UserFactory.INSTANCE;
    }

    public static UserAccessInterface getUserDAL()
    {
        return new UserDAL();
    }
}
```

```
public final class UserFactory {

    private UserDAL userDAL;

    public UserDAL getUserDAL() {
        return userDAL;
    }

    public void setUserDAL(UserDAL userDAL) {
        this.userDAL = userDAL;
    }
}
```

IoC & DIP & DI

```java
public class DependencyInjectionListener implements ServletContextListener{
    @Override
    public void contextInitialized(ServletContextEvent sce) {

        // Creating UserFactory instance
        UserFactory userFactory = new UserFactory();

        // Injecting UserDAL instance into userFactory Instance
        userFactory.setUserDAL(new UserDAL());

        // Injecting userFactory instance into AbstractResource
        AbstractResource.setUserFactory(userFactory);
    }
}
```

IoC & DIP & DI

UNIVERSITÉ CÔTE D'AZUR

- We are done !
- DependencyInjectionListener will be created at server application start
  - Will create a UserFactory instance
  - Will provide UserDAL instance into UserFactory
  - Will provide this instance to AbstractResource
- User is no more linked to UserFactory Implementation
- Thankfully Dependency Injection is a lot easier with @AutoWired !

Recap

- IoC **Architectural** design pattern / design principle
  - Helps designing loosely coupled systems by inverting controls
- Dependency Inversion **Principle**
  - Helps designing loosely coupled systems using abstraction
- Dependency Injection **Design Pattern**
  - How to achieve loose coupling
  - Rely on other design patterns to achieve this goal